ZHEJIANG UNIVERSITY COLLEGE OF INFORMATION SCIENCE AND ELECTRONICS ENGINEERING

HOMEWORK #2

Artificial Intelligence

Name: Student ID:
Major: Electronic Science and Technology

Date: 2025 年 4 月 5 日

Problem 1.

AC-3 puts back on the queue every arc (X_k, X_i) whenever any value is deleted from the domain of X_i, even if each value of X_k is consistent with several remaining values of X_i. Suppose that, for every arc (X_k, X_i), we keep track of the number of remaining values of X_i that are consistent with each value of X_k. Explain how to update these numbers efficiently and hence show that arc consistency can be enforced in total time $O(n^2 d^2)$.

Answer :

To efficiently update the number of remaining values of X_i that are consistent with each value of X_k, we can use the following approach:

Step 1: Initialization

For every arc (X_k, X_i), initialize a data structure to store the count of remaining values of X_i that are consistent with each value of X_k. This can be done by iterating through all pairs of values in the domains of X_k and X_i and checking their consistency. The time complexity for this initialization is $O(n^2 d^2)$, where n is the number of variables and d is the maximum domain size.

Step 2: Propagation

When a value is removed from the domain of X_i, update the counts for all arcs (X_k, X_i) where X_k is a neighbor of X_i. Specifically: For each value $v_k \in \text{Domain}(X_k)$, decrement the count of consistent values in X_i if the removed value from X_i was consistent with v_k. If the count for v_k becomes zero, remove v_k from the domain of X_k and propagate this change to the neighbors of X_k.

Step 3: Efficiency

To ensure efficiency, maintain a queue of arcs to be processed. Each time a value is removed from a domain, add the affected arcs to the queue. Process each arc in the queue by updating the counts and propagating changes as necessary.

Time Complexity Analysis:

Each arc (X_k, X_i) is processed at most d times, as each value in the domain of X_i can be removed at most once. For each arc processing, we iterate over at most d values in the domain of X_k, leading to a complexity of $O(d)$ per arc processing. Since there are $O(n^2)$ arcs and each arc is processed $O(d)$ times, the total time complexity is $O(n^2d^2)$.

Problem 2.

Consider the problem of placing k knights on an $n \times n$ chessboard such that no two knights are attacking each other, where k is given and $k \leq n^2$.

a. Choose a CSP formulation. In your formulation, what are the variables?

b. What are the possible values of each variable?

c. What sets of variables are constrained, and how?

d. Now consider the problem of putting as many knights as possible on the board without any attacks. Explain how to solve this with local search by defining appropriate Actions and Result functions and a sensible objective function.

Answer : 2

a. Variables: The variables in the CSP formulation are the positions of the knights on the chessboard. Let $X_1, X_2, \ldots, X_k$ represent the positions of the k knights, where each variable X_i corresponds to the position of the i-th knight.

b. Possible Values: The possible values for each variable X_i are the coordinates of the cells on the $n \times n$ chessboard. Specifically, the domain of each variable is:

$$\text{Domain}(X_i) = \{(r, c) \mid 1 \leq r, c \leq n\}$$

where r and c represent the row and column indices of the chessboard.

c. Constraints: The constraints ensure that no two knights attack each other. A knight attacks another knight if it can move to its position in one legal knight move. For any two variables $X_i = (r_i, c_i)$ and $X_j = (r_j, c_j)$, the constraint is:

$$|r_i - r_j| = 2 \wedge |c_i - c_j| = 1 \quad \text{or} \quad |r_i - r_j| = 1 \wedge |c_i - c_j| = 2$$

Besides, we also need to ensure that all knights are placed on different cells:

$$r_i \neq r_j \wedge c_i \neq c_j$$

This constraint must hold for all pairs of knights i and j ($i \neq j$).

d. Local Search for Maximum Knights:

Actions: An action involves adding a knight to an empty cell, removing a knight from a cell, or moving a knight from one cell to another.

Result: The result of an action is a new configuration of knights on the chessboard.

Objective Function: The objective function is to maximize the number of knights placed on the board while satisfying the constraints. Let N be the number of knights on the board, and let C be the number of constraint violations. The objective function can be defined as:

$$f = N - \lambda C$$

where λ is a penalty factor for constraint violations.

Algorithm: Use a hill-climbing or simulated annealing algorithm to iteratively improve the configuration: 1. Start with an empty board or a random valid configuration. 2. At each step, choose an action that improves the objective function. 3. If no improving action exists, terminate the search.

Problem 3.

Consider the following sentence:

$$[(Food \Rightarrow Party) \vee (Drinks \Rightarrow Party)] \Rightarrow [(Food \wedge Drinks) \Rightarrow Party].$$

a. Determine, using enumeration, whether this sentence is valid, satisfiable (but not valid), or unsatisfiable.

b. Convert the left-hand and right-hand sides of the main implication into CNF, showing each step, and explain how the results confirm your answer to (a).

c. Prove your answer to (a) using resolution.

Answer : a. :

We evaluate all possible truth assignments for F, D, and P. The truth table is as follows:

F	D	P	$F \Rightarrow P$	$D \Rightarrow P$	$(F \Rightarrow P) \vee (D \Rightarrow P)$	$(F \wedge D) \Rightarrow P$
T	T	T	T	T	T	T
T	T	F	F	F	F	F
T	F	T	T	T	T	T
T	F	F	F	T	T	T
F	T	T	T	T	T	T
F	T	F	T	F	T	T
F	F	T	T	T	T	T
F	F	F	T	T	T	T

From the table, the sentence evaluates to true for all possible truth assignments. Therefore, the sentence is valid.

b. Conversion to CNF: We convert both the left-hand side (LHS) and right-hand side (RHS) of the main implication into Conjunctive Normal Form (CNF).

1. Left-hand side (LHS):

$$(F \Rightarrow P) \vee (D \Rightarrow P)$$

Using the equivalence $A \Rightarrow B \equiv \neg A \vee B$, we rewrite:

$$(\neg F \vee P) \vee (\neg D \vee P)$$

$$(\neg F \vee P \vee \neg D \vee P)$$

$$(\neg F \vee \neg D \vee P)$$

2. **Right-hand side (RHS):**

$$(F \wedge D) \Rightarrow P$$

Using the equivalence $A \Rightarrow B \equiv \neg A \vee B$, we rewrite:

$$\neg(F \wedge D) \vee P$$

Using De Morgan's laws:

$$(\neg F \vee \neg D) \vee P$$

Distribute the disjunction:

$$\neg F \vee \neg D \vee P$$

The CNF for the entire sentence is:

$$(\neg F \vee \neg D \vee P) \Rightarrow (\neg F \vee \neg D \vee P)$$

c. Proof Using Resolution: To prove the validity of the sentence using resolution, we negate the sentence and show that it leads to a contradiction.

1. Negate the sentence:

$$\neg\left([(F \Rightarrow P) \vee (D \Rightarrow P)] \Rightarrow [(F \wedge D) \Rightarrow P]\right)$$

This is equivalent to:

$$[(F \Rightarrow P) \vee (D \Rightarrow P)] \wedge \neg\,[(F \wedge D) \Rightarrow P]$$

2. Simplify each part:

$(F \Rightarrow P) \vee (D \Rightarrow P)$ becomes $\neg F \vee \neg D \vee P$.

$\neg\,[(F \wedge D) \Rightarrow P]$ becomes $(F \wedge D) \wedge \neg P$.

3. Combine into CNF:

$$(\neg F \vee \neg D \vee P) \wedge (F \wedge D \wedge \neg P)$$

from the CNF, we can see that the sentence is unvalid, because there is no way to satisfy both $(\neg F \vee \neg D \vee P)$ and $(F \wedge D \wedge \neg P)$ simultaneously.

Thus the original sentence is valid.

Problem 4.

Prove each of the following assertions:

(a) α is valid if and only if $True \models \alpha$.

(b) For any α, $False \models \alpha$.

(c) $\alpha \models \beta$ if and only if the sentence $(\alpha \Rightarrow \beta)$ is valid.

(d) $\alpha \equiv \beta$ if and only if the sentence $(\alpha \Leftrightarrow \beta)$ is valid.

(e) $\alpha \models \beta$ if and only if the sentence $(\alpha \wedge \neg\beta)$ is unsatisfiable.

Answer: (a) α is valid if and only if $True \models \alpha$: A sentence α is valid if it is true under all possible interpretations. The statement $True \models \alpha$ means that α is true in every model where $True$ is true (which is all models). Therefore, α is valid if and only if $True \models \alpha$.

(b) For any α, $False \models \alpha$: The statement $False \models \alpha$ means that α is true in every model where $False$ is true. Since $False$ is never true in any model, the condition is vacuously satisfied. Thus, for any α, $False \models \alpha$.

(c) $\alpha \models \beta$ if and only if the sentence $(\alpha \Rightarrow \beta)$ is valid: The statement $\alpha \models \beta$ means that in every model where α is true, β is also true. The sentence $(\alpha \Rightarrow \beta)$ is valid if it is true in all models. By the definition of implication, $(\alpha \Rightarrow \beta)$ is true whenever α is false or β is true. This is equivalent to saying that $\alpha \models \beta$. Therefore, $\alpha \models \beta$ if and only if $(\alpha \Rightarrow \beta)$ is valid.

(d) $\alpha \equiv \beta$ if and only if the sentence $(\alpha \Leftrightarrow \beta)$ is valid: The statement $\alpha \equiv \beta$ means that α and β have the same truth value in every model. The sentence $(\alpha \Leftrightarrow \beta)$ is valid if it is true in all models. By the definition of equivalence, $(\alpha \Leftrightarrow \beta)$ is true if α and β have the same truth value. Therefore, $\alpha \equiv \beta$ if and only if $(\alpha \Leftrightarrow \beta)$ is valid.

(e) $\alpha \models \beta$ if and only if the sentence $(\alpha \wedge \neg\beta)$ is unsatisfiable: The statement $\alpha \models \beta$ means that in every model where α is true, β is also true. The sentence $(\alpha \wedge \neg\beta)$ is unsatisfiable if there is no model where α is true and β is false. This is equivalent to saying that $\alpha \models \beta$. Therefore, $\alpha \models \beta$ if and only if $(\alpha \wedge \neg\beta)$ is unsatisfiable.